Documento de arquitectura

1. Diagrama hexagonal



El dominio define los puertos, los adaptadores los cumplen y las dependencias apuntan hacia adentro. Los adaptadores de entrada traducen lo que llega de afuera, y los de salida implementan lo que el dominio necesita.

2. Puertos y adaptadores

Puerto (s)	Adaptador (es)	Tipo
RegistrarLecturaPort	api_dispositivos.py	Entrada
ConsultarEstadoPort, ConsultarHistorialPort, ListarPlantasPort, DiagnosticarMedicionPort, ListarEspeciesPort, RegistrarUsuarioPort, IniciarSesionPort, RegistrarPlantaPort	api_front.py	Entrada
TablaReferenciaPort, CatalogoEspeciesPort	tabla_csv.py, tabla_bd.py	Salida
HistorialLecturasPort	historial_bd.py	Salida

PlantaRepositorioPort, UsuarioRepositorioPort	cuentas_bd.py	Salida
DispositivoRepositorioPort	dispositivos_bd.py	Salida
HasheadorClavePort	hasheador.py	Salida
EmisorTokenPort	emisor_jwt.py	Salida

3. Dónde se emite y dónde se verifica el JWT

El JWT se emite en el dominio a través del puerto *EmisorTokenPort*. Primero *IniciarSesion* pide un token de usuario y *RegistrarPlanta* pide el token del dispositivo, creando se en *emisor_jwt.py*. **Y se verifica** en el adaptador de entrada seguridad.py, con los decoradores *requiere_usuario* y *requiere_dispositivo*, el verificador se inyecta desde app.py.

4. Principios SOLID en el código

Inversión de dependencias (D). El caso de uso depende de interfaces que declara el dominio, y quien decide la implementación es app.py.

dominio/ puertos/entrada.py

```python
class RegistrarLecturaPort(ABC):
    @abstractmethod
    def ejecutar(self, dispositivo_id: str, lectura: Lectura) -> Diagnostico:
        raise NotImplementedError
```

dominio/servicios/registrar_lectura.py

```python
class RegistrarLectura(RegistrarLecturaPort):
    def __init__(
        self,
        dispositivos: DispositivoRepositorioPort,
        plantas: PlantaRepositorioPort,
        tabla: TablaReferenciaPort,
        historial: HistorialLecturasPort,
        servicio: ServicioDiagnostico,
    ):
        self._dispositivos = dispositivos
        self._plantas = plantas
        self._tabla = tabla
        self._historial = historial
        self._servicio = servicio
```

app.py

```
    app.register_blueprint(
        api_dispositivos.crear_blueprint(
            seguridad, RegistrarLectura(dispositivos, plantas, tabla, historial, servicio)
        )
    )
    return app
```

Abierto/cerrado (O). Cambiar CSV por BD o agregar el ESP32 es agregar un adaptador, sin modificar el dominio.

```
def crear_app(
    tipo_tabla: str = None, ruta_bd: str = None, secreto: str = None
) -> Flask:
    tipo_tabla = (tipo_tabla or os.environ.get("MATERA_TABLA", "csv")).lower()
    ruta_bd = ruta_bd or os.environ.get("MATERA_BD", os.path.join(_AQUI, "matera.db"))
    secreto = secreto or os.environ.get("MATERA_SECRETO", "secreto-solo-para-desarrollo-cambi

    bd.crear_tablas(ruta_bd)

    # Puertos de salida
    tabla_csv = TablaCSV(_RUTA_CSV)
    if tipo_tabla == "bd":
        tabla = TablaBD(ruta_bd)
        if tabla.esta_vacia():
            tabla.cargar(tabla_csv.listar_todas())
    elif tipo_tabla == "csv":
        tabla = tabla_csv
    else:
        raise ValueError(f"MATERA_TABLA debe ser 'csv' o 'bd', no {tipo_tabla!r}")
```

Responsabilidad única (S). El adaptador del dispositivo solo traduce y revisa el alcance del token.

```
"""API para los dispositivos (el ESP32 físico o el simulador). Solo
acepta lecturas, y solo de la planta a la que pertenece el token."""

from flask import Blueprint, g, jsonify, request

from adaptadores.entrada import dtos
from adaptadores.entrada.errores import respuesta_error
from adaptadores.entrada.seguridad import Seguridad
from dominio.puertos.entrada import RegistrarLecturaPort


def crear_blueprint(seguridad: Seguridad, registrar_lectura: RegistrarLecturaPort) -> Blueprint:
    bp = Blueprint("api_dispositivos", __name__, url_prefix="/api/dispositivos")

    @bp.route("/<dispositivo_id>/lecturas", methods=["POST"])
    @seguridad.requiere_dispositivo
    def lecturas(dispositivo_id):
        # El token de un dispositivo no sirve para reportar por otro.
        if g.dispositivo_id != dispositivo_id:
            return respuesta_error(
                "DISPOSITIVO_NO_AUTORIZADO", "El token no corresponde a este dispositivo", 403
            )
        lectura = dtos.lectura_desde_request(request.get_json(silent=True) or {}, dispositivo_id)
        diagnostico = registrar_lectura.ejecutar(dispositivo_id, lectura)
        return jsonify({"estado": diagnostico.estado.value}), 201

    return bp
```

5. Prueba de cambio

Commit que agrega el adaptador de dispositivos y el simulador: **[pegar aquí el enlace al commit “Agregar adaptador de dispositivos y simulador del ESP32”]**.

Ese commit toca tres archivos: api_dispositivos.py, simulador_esp32.py y app.py, que es la composición y no el dominio. No modifica ningún archivo de servidor/dominio/. Se comprueba con git show --stat <hash>. El script utilidades/crear_historial_git.py arma los commits en ese orden y falla si el del dispositivo toca el dominio. CSV y BD se intercambian con MATERA_TABLA=csv o MATERA_TABLA=bd.

6. Qué cambiaría si las lecturas llegaran por MQTT

Si las lecturas llegan a MQTT, se agregaría un nuevo adaptador de entrada que se suscribe a *matera/dispositivos/+/lecturas*, convierte el mensaje con la función *lectura_desde_request* de *dtos.py* y llama a *RegistrarLecturaPort*.ejecutar, igual que *api_dispositivos.py.* Además de modificar algunos archivos como la *app.py* para algunas lineas, el *requirements.txt* con la librería cliente de MQTT, el adaptador *seguridad.py* con las credenciales del broker o un token en el mensaje, mientras que el resto el dominio, los puertos y los adaptadores de salida no cambian. MQTT es otra forma de entrada, así que se agrega un adaptador de entrada.